Practicum: Bepaling van de brandpuntsafstand van een onbekende lens

Leerdoelen

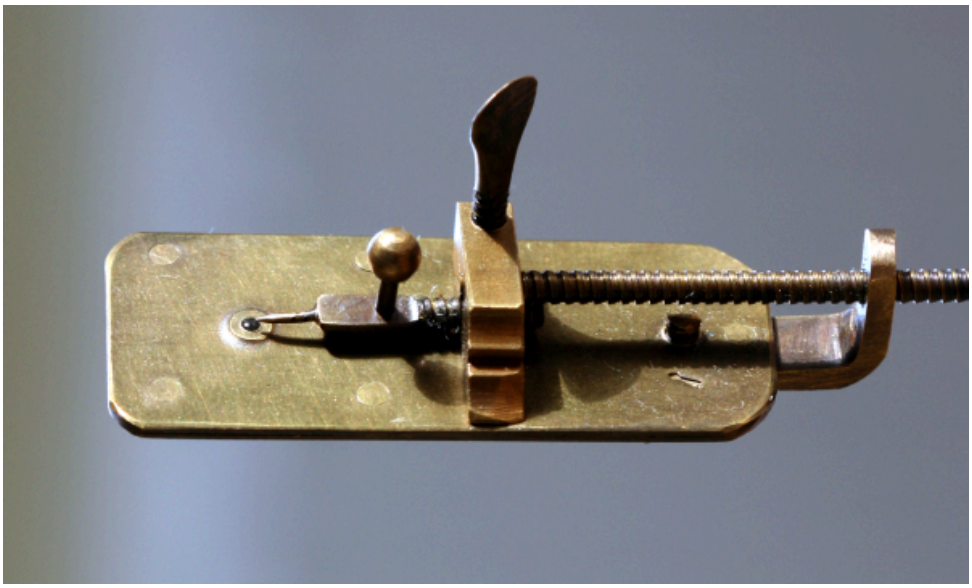
- ☐ Begrip van het belang van het gebruik van een (digitaal) labjournaal bij natuurkundig onderzoek.
- ☐ Zelfstandig kunnen bijhouden van een (digitaal) labjournaal in natuurkundepractica.
- ☐ Begrip van en toepassen van de lenzenformule.
- ☐ Basisvaardigheid in datamanipulatie en visualisatie in Python.

Veiligheid

1. We werken dit practicum met lenzen en LED lampen. Ga voorzichtig om met de lenzen en pas op voor de LED lampen, deze kunnen heet worden en mogen niet warm opgeborgen worden.
2. De LED lamp kan maximaal 12 volt aan!

Introductie

Al sinds de oudheid gebruiken mensen lenzen om beelden te vergroten of scherp te stellen. De oude Grieken en Romeinen kenden al glaskogeltjes die het licht bundelden, en in de middeleeuwen verschenen de eerste brillen. In de 17e eeuw werden brillen en lenzen steeds verfijnder, en in die tijd begon men microscopen en telescopen te ontwikkelen. Een van de bekendste Nederlanders, Antoni van Leeuwenhoek, bouwde met zelfgeslepen lenzen ontzettend sterke microscopen[1], zie Fig. 1.



Figuur 1: Een replica van één van de eerste microscopen van Antoni van Leeuwenhoek.

Met deze microscopen werden voor het eerst bacteriën, spiervezels en bloedcellen ontdekt. Zijn werk markeert het begin van de microscopie, en laat zien hoe een relatief eenvoudige lens tot grote wetenschappelijke ontdekkingen kan leiden.

De werking van een lens is in essentie eenvoudig: een lens buigt lichtstralen af doordat het licht bij de overgang van lucht naar glas (of andersom) van richting verandert. Hierdoor kunnen lichtstralen die uit één punt van een voorwerp komen, weer in één punt samenkomen: het beeldpunt. Hoe sterk een lens lichtstralen bundelt of spreidt, wordt bepaald door de *brandpuntsafstand* f.

De wiskundige relatie tussen voorwerpafstand v, beeldafstand b en brandpuntsafstand f staat bekend als de *lenzenformule*:

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}.$$

In dit practicum ga je deze relatie zelf onderzoeken. Door voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b te meten, kun je de brandpuntsafstand van een onbekende lens afleiden. Op deze manier volg je in zekere zin de voetsporen van Van Leeuwenhoek: met een lens de wereld op een nieuwe manier zichtbaar maken.

Voorbereidingsopdracht 1

1. Neem een laptop mee.
2. Zorg voor een goede installatie van Python & Jupyter notebooks, zie practicumhandleiding week 1.

[1] Het vakmanschap van Antoni van Leeuwenhoek was zo uitzonderlijk, dat het tot voor kort een raadsel bleef hoe hij zulke krachtige microscopen kon bouwen. Pas enkele jaren geleden is dit mysterie deels ontrafeld in de nucleaire reactor in Delft (zie dit artikel).

Opdracht 1(a)

Meet voor verschillende voorwerpsafstanden v de bijbehorende beeldafstanden b en vergrotingen.

1. Noteer bij elke meting ook de meetfout σ_b.
2. Leg uit hoe je de grootte van σ_b hebt bepaald. Hoe kun je deze nauwkeuriger bepalen?
3. Noteer deze gegevens in je Digitaal Labjournaal.

Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Data invoeren

```
import numpy as np

# meetwaarden
b = np.array([])
v = np.array([])
b_err = np.array([])
```

Opdracht 1(b)

Maak in Python een grafiek waarin je b uitzet tegen v. Voeg duidelijke labels en eenheden toe.
Tip! Gebruik de onderstaande code en plak deze in je labjournaal:

Grafiek maken

```
import matplotlib.pyplot as plt

# plotten
plt.errorbar(v, b, yerr=b_err, fmt='o')
plt.xlabel("v (m)")
plt.ylabel("b (m)")
plt.show()
```

Zoals je kunt zien in het figuur is er een duidelijk, maar soms ook verwarrend, verbanden tussen de beeldafstand en de voorwerpsafstand. De lenzenformule

$$\frac{1}{f} = \frac{1}{v} + \frac{1}{b}, \tag{1}$$

geeft echter niet direct het verband aan tussen b en v.

Opdracht 1(c)

1. Herschrijf de lensvergelijking zodat b wordt uitgedrukt als functie van v en f.
2. Gebruik onderstaande Python-code en vul deze correct aan.
3. Noteer je afleiding in je Digitaal Labjournaal.

Functie maken

```
def beeldafstand(v, f):
    return (...)
```

Opdracht 1(d)

1. Maak een array die loopt van 0 tot 1 meter in stapjes van 1 cm.
2. Teken de functie van de *beeldafstand* voor verschillende waarden van de brandpuntsafstand f. Welke waarde komt het meest overeen met de onbekende lens?

Lineaire span

```
v_fit = np.linspace(0, 1, 101) # in meters
# Gebruik voor elke gekozen f alleen v_fit > f (reeel beeld).
# Bij v = f is de beeldafstand niet eindig.
```

Het tekenen van verschillende waarden van *f* levert een eerste schatting op, maar houdt geen rekening met meetonzekerheden. Grotere afstanden betekenen niet automatisch nauwkeurigere metingen: schat de onzekerheid per meting. We gebruiken hier *lineariseren*: door zowel de beeldafstand als de voorwerpsafstand te inverteren, ontstaat een lineair verband tussen de nieuwe variabelen. Lineariseren is niet hetzelfde als inverteerbaarheid. Bij deze transformatie moeten ook de onzekerheden worden omgerekend.

Opdracht 1(e)

1. Maak een nieuwe grafiek waarbij je $1/b$ uitzet tegen $1/v$.
2. Schrijf de lenzenformule om in de vorm
$$y = ax + c, \qquad y = \frac{1}{b}, \quad x = \frac{1}{v}. \tag{2}$$
3. Maak een nieuwe functie genaamd *lineair*, met als invoer `inverse_v = 1/v` en parameter `f`. De uitvoer is `1/b`.
4. Wat valt je op aan de grafiek? Wat is de betekenis van het snijpunt met de y-as?
5. Voeg nu enkele lijnen toe met verschillende waarden van de brandpuntsafstand f.

Functie maken

```
def lineair(inverse_v, f):
    return (...)
```

Zoals je misschien hebt opgemerkt, ziet de grafiek er nu aanzienlijk beter uit. De vraag blijft echter wat het snijpunt met de y-as betekent en welke waarde hier precies bij hoort. Om dit op een natuurkundig correcte manier uit te werken, gebruiken natuurkundigen vaak modellen. In dit geval is dat natuurlijk de functie $lineair$.
Nu moeten we ons model nog toetsen (*fitten*!) aan de gemeten data. Dit is in principe een optimalisatieprobleem: we moeten de fitwaarde *f* zo kiezen dat de afwijkingen tussen model en meting zo klein mogelijk zijn[2]. Hiervoor maken we gebruik van de rekenkracht van Python, met de wetenschappelijke module **scipy**.

Data fitten

```
from scipy.optimize import curve_fit

# Schatting van parameters
p0 = [1] # eerste schatting van f, 1 meter

# aanroepen fit
popt, pcov = curve_fit(lineair, 1/v, 1/b, sigma=(...), p0=p0,
                       absolute_sigma=True)

# resultaat printen
print("$f$ =", popt, "+-", np.sqrt(np.diag(pcov)))
```

[2] Het maakt niet uit als deze stap nog onduidelijk is; het is een trucje dat tijdens wiskunde 2.1 verder wordt uitgelegd.

Opdracht 1(f)

1. Bereken de fout op $\frac{1}{b}$, gebruik hiervoor de relatieve fout van b. Als dit niet lukt, sla deze stap dan over.
2. Bepaal f door middel van een fit, gebruik hiervoor de *scipy*-module. Mocht het bepalen van de fout in de vorige opdracht niet gelukt zijn, verwijder dan de argumenten `sigma=(...)` en `absolute_sigma=True`. De onzekerheid wordt dan geschat uit de residuen, onder de aanname van gelijke spreiding in `1/b`.
3. Wat is de gevonden waarde voor *f*, en welke onzekerheid hoort hierop?

Afronding

1. Zet de gevonden figuren in je digitaal labjournaal en maak een duidelijk onderschrift waarin je uitlegt wat de figuren laten zien.
2. Vul de **Resultaat- en Discussie**-sectie aan in je labjournaal. Hierin moet de uiteindelijke waarde van f (met onzekerheid) staan en een korte discussie of deze waarde **realistisch** en **betrouwbaar** is.

Appendix: data fitten met `curve_fit`

Wanneer je meetpunten hebt (x_i, y_i) met meetfouten σ_i, wil je vaak een model $y(x;\theta)$ vinden dat de data zo goed mogelijk beschrijft. Hierbij zijn θ de onbekende parameters (bijvoorbeeld een brandpuntsafstand f, of de richtingscoëfficiënt a en het intercept b van een lijn).

Least squares-principe.

Het idee van een least squares-fit is dat je de afwijkingen tussen model en meting (de residuen) minimaliseert:

$$r_i = y_i^{(\text{gemeten})} - y(x_i;\theta).$$

Zonder meetfouten.

Als de individuele meetonzekerheden σ_i niet bekend zijn, minimaliseer je de som van de kwadraten van de residuen:

$$S(\theta) = \sum_i r_i^2.$$

Dit heet een ordinary least squares-fit (OLS): alle meetpunten wegen even zwaar mee.

Met meetfouten.

Als de onzekerheden σ_i wel bekend zijn, deel je de residuen door de fout en minimaliseer je

$$\chi^2(\theta) = \sum_i \left(\frac{r_i}{\sigma_i}\right)^2.$$

Dit heet een weighted least squares-fit (WLS) of χ^2-fit. Bij onafhankelijke, normaal verdeelde meetfouten met bekende standaardafwijkingen komt dit overeen met maximum-likelihoodschatting, als het model correct is en de onzekerheid in x verwaarloosbaar is.

Uitkomst.

De functie `curve_fit` levert:

- de optimale parameters $\hat{\theta} =$ `popt`;
- de covariantiematrix $C =$ `pcov`, die een schatting geeft van de onzekerheden en correlaties van de parameters.

De standaardfout van parameter θ_j volgt uit

$$\sigma_{\theta_j} = \sqrt{C_{jj}}.$$

In Python komt dit overeen met:

- geen `sigma` meegeven $\rightarrow$ ordinary least squares (som van gekwadrateerde residuen);
- wel `sigma` meegeven $\rightarrow$ weighted least squares (χ^2).

Interpretatie.

De kwaliteit van de fit hangt af van:

- het gekozen model $y(x;\theta)$ (is het fysisch correct en past het bij de data?);
- de spreiding van de meetpunten en hun onzekerheden σ_i;
- de initiële schatting p_0 voor de parameters: bij een niet-lineaire fit kan een slechte start leiden tot lokale minima.

Een directe fit aan de oorspronkelijke meetwaarden voorkomt dat je de data en hun onzekerheidsverdeling hoeft te transformeren. Ook een directe fit vereist een geschikt model, realistische onzekerheden en een bruikbare startschatting; `curve_fit` garandeert geen globaal optimum. Bij de linearisatie geldt voor kleine relatieve onzekerheden: `sigma_inverse_b = b_err / b**2`. De fit hieronder behandelt de x-waarden als exact.

Voorbeeld: fitten met `curve_fit`

```python
import numpy as np
from scipy.optimize import curve_fit

# Modeldefinitie (hier: rechte lijn)
def linear(x, a, b):
    return a * x + b

# Meetdata (x,y) met fouten
xdata = np.array([1,2,3,4,5])
ydata = np.array([1.1,1.9,3.2,3.9,5.1])
yerr  = np.array([0.1]*5)

# Eerste schatting van parameters a,b
p0 = [1, 0]

# Fit aanroepen
popt, pcov = curve_fit(linear, xdata, ydata, sigma=yerr, p0=p0,
                       absolute_sigma=True)

# Resultaten
perr = np.sqrt(np.diag(pcov))
print("a =", popt[0], "+-", perr[0])
print("b =", popt[1], "+-", perr[1])
```

Appendix: Materialen Lijst Practica

- Onbekend Lens (1x)
- Optische rail
- Dia-houders + dia (letter F)
- Projectiescherm
- Rolmaat
- Ruiters 3 hoog, 1 laag.
- LED bron, 2 kabels en voedingsbron.